

Final Project: Stacking

 Author: 111511198 鄭恆安

Performance of Stacking Model

K- Fold Scoring = 0.6881178493836827

```
Training base learner 1/5: 5it [00:00, 8.31it/s] | 9/10 [03:48<00:22, 22.70s/it]
Training base learner 2/5: 5it [00:12, 2.50s/it]
Training base learner 3/5: 5it [00:04, 1.13it/s]
Training base learner 4/5: 5it [00:00, 130.50it/s]
Training base learner 5/5: 5it [00:06, 1.34s/it]
Accuracy: 0.8064516129032258it [00:06, 1.35s/it]
F1: 0.7857142857142857
MCC: 0.6092436974789915
100%| | 10/10 [04:13<00:00, 25.38s/it]
Kfold scoring: 0.6881178493836827
```

Stacking Model Code Explanation

Initialization

```
class StackingClassifier:
    def __init__(self,
        base_learners=[
            ('decision_tree', DecisionTreeClassifier()),
            ('knn', KNearestNeighborClassifier()),
            ('naive_bayes', NaiveBayesClassifier()),
            ('mlp', MLPClassifier())
        ],
        meta_learner = LogisticRegressionClassifier(),
        n_splits = 5,
    ):
        """
        Stacking Classifier
        :param base_learners: List of base learner models.
        :param meta_learner: Meta-learner model.
        :param n_splits: Number of splits for cross-validation.
        """

        self.base_learners = base_learners
        self.meta_learner = meta_learner
        self.n_splits = n_splits
```

- **Base Learners:** A list of tuples where each tuple contains a name and an instance of a base model (Decision Tree, k-NN, Naive Bayes, MLP).
- **Meta Learner:** A classifier (default: Logistic Regression) that learns from the predictions of the base learners.
 - why choose Logistic Regression as meta learner
 - **Logistic Regression** is ideal for simplicity and robust performance without overfitting.
- **n_splits:** Sets the number of folds for cross-validation (default is 5).

Training StackingCVClassifier

The **StackingCVClassifier** improves upon the traditional stacking approach by introducing **cross-validation** directly into the meta-learner training process.

This refinement ensures that the meta-learner is trained on **out-of-sample predictions**, reducing the risk of overfitting.

- **Base Learners Training:**
 - Base learners are trained multiple times, each time on different folds of the dataset.
 - Their predictions on the unseen folds are collected.
- **Meta-Learner Training:**
 - The combined predictions from all base learners serve as features for the meta-learner.
 - This ensures the meta-learner is trained on predictions that simulate real test-time behavior.

Advantages of StackingCVClassifier

1. **Prevents Overfitting:** By using out-of-sample predictions, it avoids data leakage between base learners and the meta-learner.
2. **Robust Training:** The cross-validation ensures that the meta-learner is exposed to a wider variety of scenarios.
3. **Better Performance:** Often achieves higher accuracy and better generalization compared to traditional stacking.

```
def fit(self, X_train, y_train):
    """
    Fit the stacking model with cross-validation.
    :param X_train: Training data (pandas DataFrame).
    :param y_train: Training labels (pandas Series).
    """
    if(isinstance(X_train, np.ndarray)):
        X_train = pd.DataFrame(X_train)
    if(isinstance(y_train, np.ndarray)):
        y_train = pd.DataFrame(y_train, columns=['label'])

    # Convert input to numpy arrays for processing
    X_train_np = X_train.values
    y_train_np = y_train.values

    self.base_learners_fitted = []
    n_samples, n_features = X_train_np.shape
    kf = KFold(n_splits=self.n_splits, shuffle=True, random_state=42)

    # Store out-of-fold predictions
    oof_predictions = np.zeros((n_samples, len(self.base_learners)))

    for i, learner in enumerate(self.base_learners):
        learner_preds = np.zeros(n_samples)

        # Training Loop
        for train_idx, val_idx in tqdm(kf.split(X_train_np)):
            # Train on training fold
            learner_clone = learner[1] # Create a new instance
            learner_clone.fit(X_train.iloc[train_idx], y_train.iloc[train_idx])

            # Predict on validation fold
            learner_preds[val_idx] = learner_clone.predict(X_train.iloc[val_idx])
            # Combined the prediction
            oof_predictions[:, i] = learner_preds
        self.base_learners_fitted.append(learner)

    # Train meta-learner on out-of-fold predictions (convert to DataFrame for consistency)
    # Combine original features with out-of-fold predictions
    oof_predictions_df = pd.DataFrame(oof_predictions, columns=[f"BaseLearner_{i}" for i in range(len(self.base_learners))])
    self.meta_learner.fit(oof_predictions_df, y_train)
```

- **Cross-Validation (CV):** The `KFold` splits the dataset into `n_splits` for training and validation.
- **Out-of-Fold (OOF) Predictions:** A 2D array to store predictions for each fold with base learner.

Training Loop:

- Each base learner is trained on the training folds and validated on the validation fold.
- Predictions on the validation fold are stored in `oof_predictions`.

Training the Meta-Learner

- The **meta-learner** uses the OOF predictions as features to learn a second-level model.

Making Predictions

```
def predict(self, X_test):
    """
    Predict using the stacking model.
    :param X_test: Test data (pandas DataFrame).
    :return: Final predictions.
    """
```

```

# Collect base learner predictions
base_preds = np.column_stack([
    learner[1].predict(X_test) for learner in self.base_learners_fitted
])
#X_test_combined = X_test.join(pd.DataFrame(base_preds, columns=[f"BaseLearner_{i}" for i in range(len(self.base_learners))]))
# Convert to DataFrame for meta-learner
base_preds_df = pd.DataFrame(base_preds, columns=[f"BaseLearner_{i}" for i in range(len(self.base_learners))])
base_preds_df.to_csv('base_preds.csv', index=False)
# Meta-learner predicts based on base learner outputs
return self.meta_learner.predict(base_preds_df)

```

- Each base learner generates predictions for the test data.
- These predictions are passed to the meta-learner for the final predictions.

Scoring Metrics

```

def predict_score(self, y_pred, y_true):
    """
    Calculate the scoring metric.
    :param y_pred: Predicted labels.
    :param y_true: True labels.
    :return: Scoring metric.
    """
    print("Accuracy:", accuracy_score(y_true, y_pred))
    print("F1:", f1_score(y_true, y_pred))
    print("MCC:", matthews_corrcoef(y_true, y_pred))
    return 0.3 * accuracy_score(y_true, y_pred) + 0.35 * f1_score(y_true, y_pred) + 0.35 * matthews_corrcoef(y_true, y_pred)

```

- Combines three metrics: **Accuracy**, **F1 Score**, and **MCC**.
- Final score: Weighted combination:
 - `0.3 * Accuracy + 0.35 * F1 + 0.35 * MCC`.

K-Fold cross validation for given model

- Splits the dataset into n folds.
- Iteratively uses one fold for validation and the remaining n-1 folds for training.
- Evaluates the model using three metrics: **Accuracy**, **F1-Score**, and **Matthews Correlation Coefficient (MCC)**.
- Returns the averaged metrics and a combined scoring value.

```

def KFold_cross_validation(X, y, n, model):
    fold_size = len(X) // n
    data = pd.concat([X, y], axis=1)
    data = data.sample(frac=1).reset_index(drop=True)

    scores = {
        'accuracy': [],
        'f1': [],
        'mcc': [],
    }
    print('k:', n)
    for i in tqdm(range(n)):
        start, end = i * fold_size, (i + 1) * fold_size
        val_data = data[start:end]
        train_data = pd.concat([data[:start], data[end:]])

        X_train, y_train = train_data.iloc[:, :-1], pd.DataFrame(train_data.iloc[:, -1])
        X_val, y_val = val_data.iloc[:, :-1], val_data.iloc[:, -1]
        model.fit(X_train, y_train)
        y_pred = model.predict(X_val)
        scores['accuracy'].append(accuracy_score(y_val, y_pred))
        scores['f1'].append(f1_score(y_val, y_pred))
        scores['mcc'].append(matthews_corrcoef(y_val, y_pred))

    for metric in scores:
        scores[metric] = np.mean(scores[metric])
    scoring = 0.3 * scores['accuracy'] + 0.35 * scores['f1'] + 0.35 * scores['mcc']

```

```
scores['scoring'] = scoring
print('Kfold scoring:', scoring)
print('\n')
return scores
```

The concept of stacking and provide the structure

Stacking is an ensemble learning technique that combines the predictions of multiple base models (base learners) to improve overall performance. The idea is to leverage the strengths of different models and mitigate their weaknesses by training a **meta-learner** that takes the outputs of the base learners as input features and predicts the final output.

Steps in Stacking

1. **Train Base Learners:**

- Multiple models (Decision Trees, k-NN, Naive Bayes, MLPClassifier) are trained independently on the same training data.

2. **Generate Predictions:**

- Each base model makes predictions on the training and/or test data. These predictions are treated as new features.

3. **Train Meta-Learner:**

- A second-level model (Logistic Regression) is trained on the predictions of the base learners to make the final prediction.

Structure of Stacking

1. **Input Layer:**

- Original features $X_1, X_2, \dots, X_n$ from the dataset.

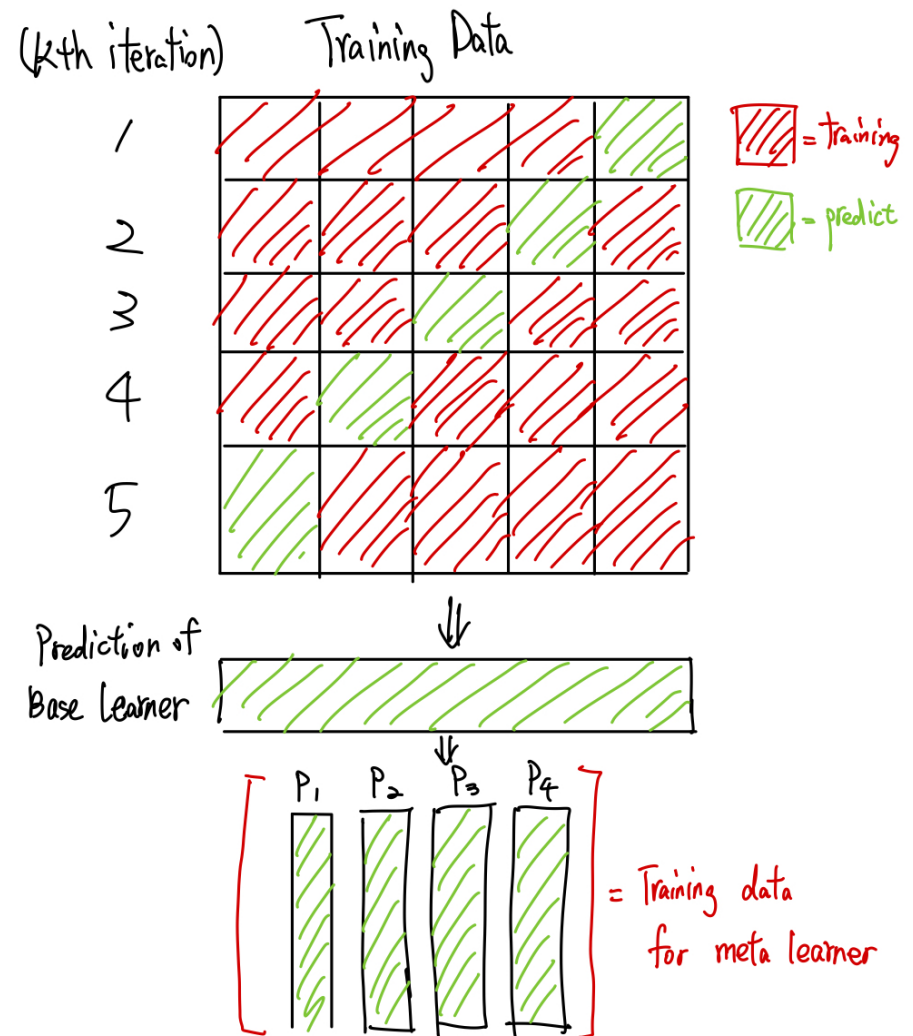
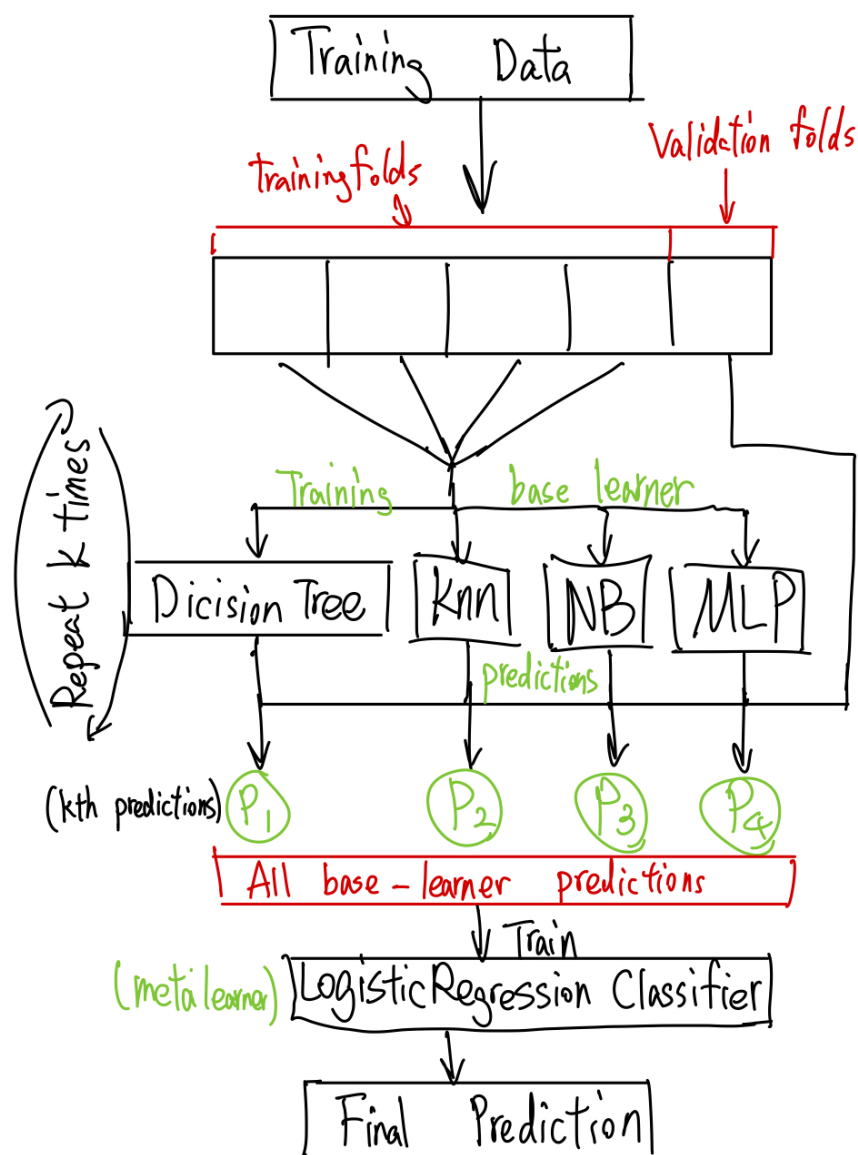
2. **Base Learners:**

- Independent models (e.g., Decision Tree, k-NN, Naive Bayes) trained on the input features.
- Parallel models trained on the same dataset.
- Outputs are predictions $P_1, P_2, \dots, P_m$.

3. **Meta-Learner:**

- A higher-level model trained on the predictions $P_1, P_2, \dots, P_m$ of the base learners.
- Produces the final output.

Visualization of Stacking Structure and Process of Generating Meta Learner Training Data



Discussion

1. Meta-Learner Selection

- **Why Logistic Regression Works Better in This Case:**
 - Logistic Regression is linear and less prone to overfitting, especially with small datasets or when the meta-feature space (predictions from base learners) is limited.
 - The simplicity of Logistic Regression allows it to effectively combine predictions from the base learners without requiring much training data.
- **Why MLP Initially Underperformed:**
 - MLPs require more data to train effectively, particularly when dealing with only a few meta-features (e.g., 4 base learner predictions).
 - Overfitting is more likely with MLPs when the feature space is small and data is limited.

2. Base Learner Selection

- **Diversity Matters:**
 - The choice of diverse base learners like Decision Trees, KNN, and Naive Bayes ensures varied decision boundaries and error patterns.
 - Including models with different assumptions (e.g., linear vs. non-linear) helps the meta-learner integrate complementary strengths.
- **Balancing Performance:**
 - It's essential to ensure that each base learner contributes positively. Weak or poorly trained base learners can degrade the stacking classifier's performance.

3. Comparison of Base Learners and Stacking Classifier

```
• heng-ancheng@Heng-AndeAir pycode % python3 main.py  
stacking 0.6881178493836827  
  
logistic_regression 0.5528294732780088  
  
decision_tree 0.7904397236092626  
  
knn 0.6799207439788169  
  
naive_bayes 0.6498236057673711  
  
mlp 0.7631213812659511
```